

CS2105 Comp. Networks Notes

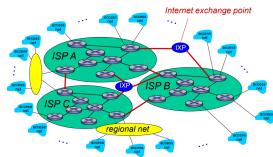
AY23/24 Sem 1. github.com/gerteck

1. Computer Networks Introduction

- Fundamental concepts and principles behind computer networking, with internet as case study.
- Connected by communication links and packet switches.

Internet

- The Internet is a network of connected computing devices (e.g. PC, server, laptop, smartphone).
- Such devices are known as hosts or end systems. Hosts run network applications (e.g. Tele, browser, Zoom)
- Packet switching network, users' packets share network resources that are used on demand. Excessive congestion is possible.
- Network of networks** Hosts connect to Internet via access ISPs (Internet Service Providers), which themselves are interconnected.



Network Edge (Access Network)

- The access network is the network that physically connects an end system to the first router on a path from that end system to any distant end system.
- E.g. Residential access networks, mobile access networks.

Network Core

- A mesh of interconnected routers.
- Data is transmitted through network through:
- Circuit switching:** dedicated circuit per call
- Packet switching:** data sent through net in discrete “chunks”

Circuit Switching

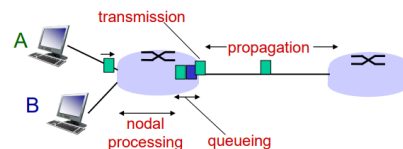
- End-end resources are allocated to and reserved for “call” between source & dest:
- call setup required, circuit-like (guaranteed) performance
- circuit segment idle **silent periods** if not used by call (no sharing)
- commonly used in traditional telephone network

Packet Switching

- Host sending function: breaks application message into smaller chunks, known as packets, of length L bits. Then, transmits packets onto the link at transmission rate R .
- Packets are passed from one router to the next, across links on path from source to destination.
- link transmission rate is aka link capacity or **link bandwidth**
- Packet transmission delay** (time taken to transmit L-bit packet into Link) = $\frac{L}{R}$
- Store-and-forward:** entire packet must arrive at a router before it can be transmitted on the next link.

Delay, Loss, Throughput

Four sources of packet delay, suffered at each node, from source to destination. (**Total nodal delay**)



- Nodal Processing Delay:** Time required to check for bit errors and determine output link. Typically $< \text{msec}$.
- Queueing Delay:** Time waiting in queue for transmission, depends on earlier arrived packets. Typically $< \text{msec}$. Additionally, possibility of packet loss if router queue full (buffer has reached finite capacity), and drops packet.
- Transmission Delay:** Time to push (last bit) of packet on wire. $d_{trans} = \frac{L}{R}$.
- Propagation Delay:** Time to propagate to next router. $d_{prop} = \frac{d}{s}$, where d is length of physical link, s propagation speed.
- End to end packet delay is total time taken for packet to travel from source to destination, consisting of the 4 delays.
- Throughput:** How many bits can be transmitted per unit time, usually measured for end-to-end communication (as opposed to bandwidth for specific link).

Internet Protocol Stack

Protocols logically organised into 5 “layers” according to purpose. (Additionally presentation and session layers not included)

- Application:** Where network applications and app-layer protocols reside. Packet here called message. Examples: HTTP, SMTP, FTP
- Transport:** Transports app-layer messages between application endpoints. Packet here called segment. Examples: TCP, UDP
- Network:** Moves packets (datagrams) from one host to another. Includes IP protocol and other routing protocols.
- Link:** Moves packet from one node to another. Packet here called frame. Example: Ethernet, WiFi
- Physical:** Moves individual bits within link-layer frame from one node to another. Link and transmission medium dependent.

2. Application Layer

Principles of Network Applications

- Client-Server:**
 - Server waits for incoming requests and provides required services to client. Easy scalability.
 - Client initiates contact with server and requests service.
- Peer-To-Peer (P2P):**
 - No dedicated server, instead it relies on direct communication between pairs of intermittently connected hosts called peers.
 - Self-scalability, each peer generates workload but adds service capacity by distributing files.

Process Communication

- Socket:** A software interface that process uses to send messages and receive messages from network. Generally a combination of IP address and port number.
- IP Address:** A 32-bit quantity, uniquely identifies host.
- Port Number:** A 16-bit integer used to identify a receiving process running in a host.

Requirements of Transport Service

- Reliable Data Transfer:** Data to sent correctly and completely vs. loss-tolerant.
- Throughput:** Bandwidth-sensitive apps may need guaranteed throughput of r bits/sec.
- Timing/Delay:** Real-time applications generally require low delays to be effective.
- Security:** Encryption, data integrity, authentication.

Transport Layer Protocols

Two main protocols for the Internet.

- Transmission Control Protocol (TCP)**
 - Reliable data transfer, Connection-oriented service: A handshake required.
 - Flow control, Congestion control: Throttle sender when network overloaded
 - Security: Can be enhanced at the app layer with Secure Sockets Layer
 - Does not provide: Timing and throughput guarantee
- User Datagram Protocol (UDP)**
 - Unreliable data transfer
 - Connectionless: No handshake
 - No flow control, no congestion control
 - Does not provide: Timing and throughput guarantee, security.

Application-Layer Protocols

An application-layer protocol defines:

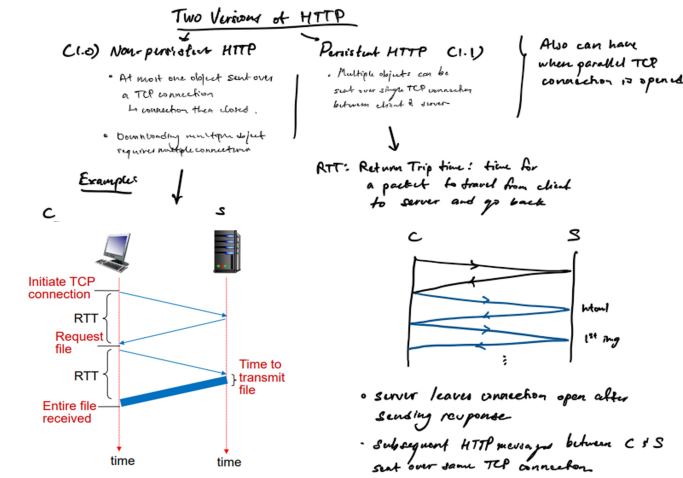
- Types of messages exchanged, e.g. request and response messages.
- Syntax of message types, e.g. fields and how they are delineated.
- Semantics of the fields, i.e. what the information means.
- Rules for when and how to send a message and respond to messages.

Web & HTTP

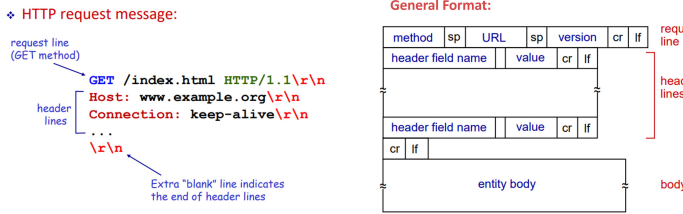
- Webpage:** Consists of base HTML files and referenced objects. Addressable by URL.
- URL made up of hostname as well as path name. (E.g. <http://www.comp.nus.edu.sg/cs2105/img/doge.jpg>)
- HyperText Transfer Protocol** is Web's app-layer protocol.
- Client-server model:** Client requests, receives and displays Web objects, server is Web server that sends objects in response.
- Stateless:** server maintains no information about clients, and **Reliable:** Over TCP.
- Three-way Handshake:** Client sends small TCP segment to ask for connection, server acknowledges and responds, client acknowledges and sends it back with request message.

HTTP versions

- **RTT**: Round trip time, time taken for packet to travel from server and back to client, does not include transmission delay.
- **Non-persistent HTTP**: Response time = 2* RTT+ file transmission time (per object)
- **Persistent HTTP**: Server leaves connection open after sending response, subsequent HTTP sent over same TCP connection. Also uses pipelining, send requests back to back.

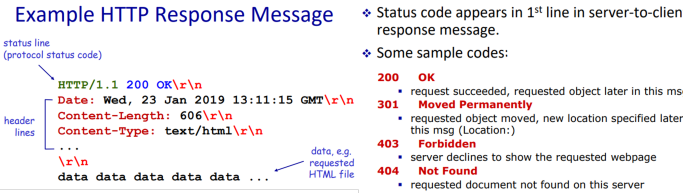


HTTP Request



- **HTTP 1.0 Methods**: GET (gets object), POST (posts form data), HEAD (gets header without body).
- **HTTP 1.1 Methods**: GET, POST, HEAD, PUT (uploads file to path specified), DELETE

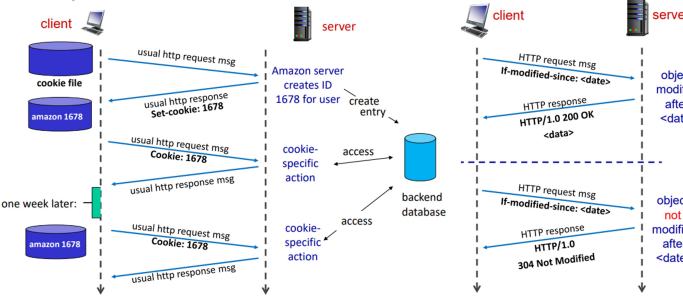
HTTP Response



Cookies

- HTTP is designed to be “stateless”, server maintains no information about past client requests.
- Good to maintain states over multiple transactions, e.g. shopping carts
- **Cookie**: http messages carry “state”:
 - 1) cookie header field of HTTP req/res messages
 - 2) cookie file kept on user host, managed by browser
 - 3) back-end database at Web site
- **Conditional GET**: In cache, specify date of cached copy in HTTP request. Server response contains no object if cached copy is up to date.

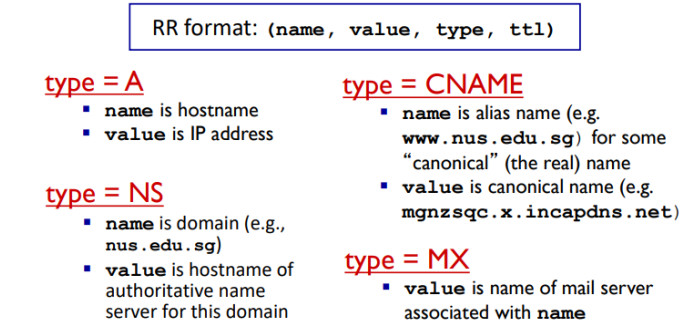
Keep User States with Cookie



Domain Name System

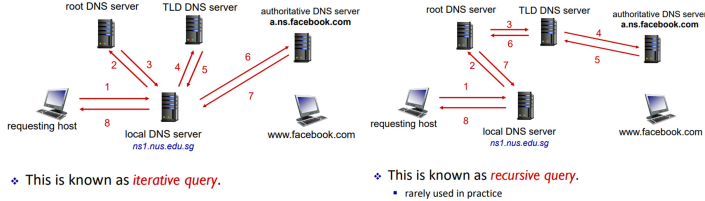
DNS translates between hostname and IP addresses. Client must carry out a DNS query to determine the IP address corresponding to the server name.

- **DNS: Resource Records (RR)**
 - ✦ Mapping between host names and IP addresses (and others) are stored as **resource records (RR)**.



- **Distributed, Hierarchical Database**: DNS servers form hierarchy to distribute mappings. Contains root servers (for Top Level Domain TLD servers), TLD servers (e.g. uk, sg), Authoritative servers.
- Local DNS servers have local cache, acts as proxy, forward query into hierarchy if answer not found locally.
- **DNS Caching**: Cache mapping, which expires after some time (TTL: time to live).
- DNS runs over **UDP**.

DNS Name Resolution



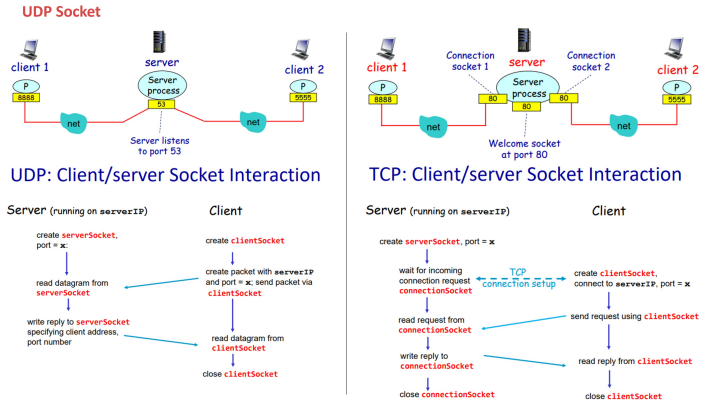
Socket Programming

Applications treat the Internet as black box, send/receive message through sockets.

- Two types of sockets
- **TCP**: reliable, byte stream-oriented socket
- **UDP**: unreliable datagram socket

UDP vs. TCP Socket

- **UDP Socket**: Sender attach des IP address + port no. to each packet. (OS inserts add. info source IP and port). Receiver extracts sender IP + port number from packet.
- **TCP Socket**: Attempts to establish TCP connection to server first. Server TCP contacted creates new socket to communicate with client, allows server to talk with multiple clients individually.



- **TCP vs. UDP Differences**
- In TCP, two processes communicate as if pipe between them. The pipe remains in place until one of two processes closes it. Sending process doesn't need to attach a destination IP / port number to the bytes in sending attempt as the logical pipe has been established
- In UDP, programmers need to form UDP datagram packets explicitly and attach destination IP address / port number to every packet.

a. Sender view of sequence numbers

b. Receiver view of sequence numbers

Key:

- Already ACK'd
- Sent, not yet ACK'd
- Out of order but already ACK'd
- Expected, not yet received
- Usable, not yet
- Not used

- **Connection oriented:** handshake before sending data. **Point to point:** one sender, one receiver. The connection is **duplex** (bidirectional data flow).
Reliable in-order.
- TCP socket is fully identified by four-tuple: (source IP addr, source port no., dest IP addr, dest port no.).
- **Multiplexing:** TCP gathers data from processes, form transport-layer segments including app data and 4-tuple pass to network layer.
- **Demultiplexing:** Connection socket created, server already noted 4-tuple. Subsequent packets directed, or demultiplexed, to the appropriate socket using those 4 values.
- TCP creates **buffers** after handshaking.

- The maximum segment size (MSS) depends on maximum transmission unit (MTU).
- Generally MSS is 1460 bytes, (MTU is 1500 bytes for Ethernet and PPP link-layer protocols.) 40 bytes split half for TCP and IP header.
- TCP Seq. no is "byte no.", first b of data in segment.
- TCP Ack. no is "seq no." of next b expected by receiver.
- Checksum computation uses 1s complement (UDP same)

The diagram illustrates the structure of a TCP segment and how data is divided into segments for transmission.

TCP Segment Structure:

- Header:** 32 bits in total, divided into two 16-bit halves.
 - First 16 bits:**
 - source port # (16 bits)
 - sequence number (32 bits)
 - acknowledgement number (32 bits)
 - checksum (16 bits)
 - Second 16 bits:**
 - dest port # (16 bits)
 - offset (4 bits)
 - reserved (6 bits)
 - control bits (2 bits)
- Body:** application data (variable length)

Annotations:

- indicate if ACK # is valid:** Points to the acknowledgement number field.
- counting byte number of data (not segment #):** Points to the sequence number field.
- TCP Sequence Number:** Points to the sequence number field.
- “Byte number” of the first byte of data in a segment.** Points to the sequence number field.
- Example: send a file of 500,000 bytes; MSS is 1,000 bytes.** Points to the sequence number field.
- SYN, FIN: Connection setup, teardown commands** Points to the control bits field.

Dividing file data into TCP segments:

The diagram shows a file of 500,000 bytes being divided into segments of 1,000 bytes each. The segments are numbered 0, 1, ..., 1,000, ..., 1,999, ..., 499,999.

Event at TCP receiver	TCP receiver action
Arrival of in-order segment with expected seq. #. All data up to expected seq. # already ACKed	Delayed ACK: wait up to 500ms for next segment. If no next segment, send ACK
Arrival of in-order segment with expected seq. #. One other segment has ACK pending	Immediately send single cumulative ACK, ACKing both in-order segments
Arrival of out-of-order segment higher-than-expected seq. # (gap detected)	Immediately send <i>duplicate</i> ACK, indicating seq. # of next expected byte
Arrival of segment that partially or completely fills gap	Immediately send ACK, provided that segment starts at expected seq. #

The diagram illustrates two scenarios of TCP timeout:

- a) Lost ACK:** The sender transmits a packet (seq=92, 8 bytes of data). The receiver receives it and sends back an ACK=100. However, the ACK packet is lost (indicated by a red 'X' and 'loss'). The sender's timer expires (indicated by a red alarm clock icon and 'timeout'), leading to a 'retransmit' of the packet.
- b) premature timeout:** The sender transmits a packet (seq=92, 8 bytes of data). The receiver receives it and sends back an ACK=100. However, the sender's timer expires (indicated by a red alarm clock icon and 'timeout') before the ACK is received. This leads to a 'retransmit' of the packet, even though the ACK was already received by the receiver.

- too long timeout: slow reaction to segment loss. Timeout interval must be longer than RTT – but RTT varies!

-
- Diagram illustrating a timeout in a Stop-and-Wait protocol:
- Sender:**
 - Sends a packet: `seq=92, 8 bytes of data`.
 - Receives `ACK 100` (triple duplicate ACKs).
 - Experiences a **timeout** (indicated by a red alarm clock icon).
 - Sends a `resend 2nd segment` packet.
 - Receiver:**
 - Receives the initial packet.
 - Sends `ACK 100` (triple duplicate ACKs).

Establishing connection

Client **Server**

Client actions:

- choose initial seq num, x
- send TCP SYN msg
- send ACK
- this segment may contain client-to-server data

Server actions:

- choose initial seq num, y
- send TCP SYNACK msg
- ACKing SYN

Handshake messages:

- SYNbit = 1, seq = x (Client to Server)
- SYNbit = 1, seq = y (Server to Client)
- ACKbit = 1, ACKnum = $x+1$ (Server to Client)
- ACKbit = 1, ACKnum = $y+1$ (Client to Server)
- seq = $x+1$ (Client to Server)

3-way handshake

Closing connection

Client **Server**

Client actions:

- clientSocket.close()
- can no longer send app data but can receive data
- wait for server close

Server actions:

- can no longer send app data

Closing messages:

- FINbit = 1, seq = u (Server to Client)
- ACKbit = 1, ACKnum = $u+1$ (Client to Server)
- FINbit = 1, seq = t (Client to Server)
- ACKbit = 1, ACKnum = $t+1$ (Server to Client)

Special Addresses	Present Use
0.0.0.0/8	Non-routable meta-address for special use
127.0.0.0/8	Loopback address. A datagram sent to an address within this block loops back inside the host. This is ordinarily implemented using only 127.0.0.1/32.
10.0.0.0/8 172.16.0.0/12 192.168.0.0/16	Private addresses, can be used without any coordination with IANA or an Internet registry.
255.255.255.255/32	Broadcast address. All hosts on the same subnet receive a datagram with such a destination address.

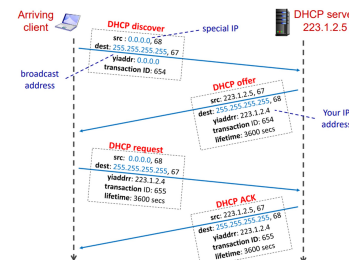
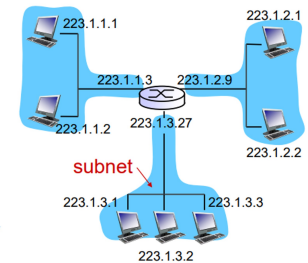


Diagram illustrating the structure of an IPv4 address (32 bits) divided into a network (subnet) prefix and a host ID.

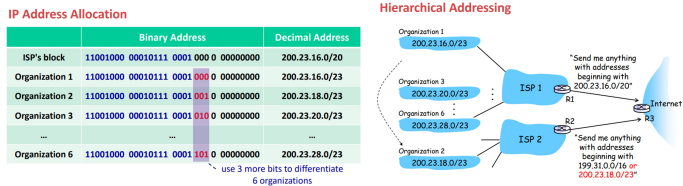
The address is shown as: 11001000 00010111 00010000 00101010

The address is divided into two parts:

- network (subnet) prefix** (n bits): 11001000 00010111 00010000
- host ID** (32 - n bits): 00101010



- **Hierarchical Addressing:** Allows efficient way of routing.

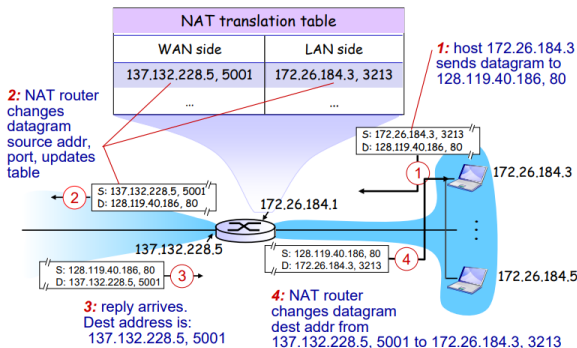


- **Longest Prefix Match:** Choose one with longer match. If IP addr matches all 23 bits for org, packet forwarded, else forwarded to latter.

Network Address Translation (NAT)

- Map IP addr space by modifying network addr info in packets IP header through traffic routing device. NAT Routers must:
- **Replace** (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #), **Remember** (in NAT translation table) the mapping and **Replace** destination fields of every incoming datagram with that stored in NAT translation table.
- **Benefits:** Single public IP for NAT router allows multiple private IP address. Change (private IP) addr of hosts in local network without notifying outside world.
- ISP change w/o changing local host addresses in local network.
- Hosts inside local network not explicitly addressable or visible by outside world (security plus).

NAT: Illustration



Routing Algorithms

- Internet: “network-of-networks”, hierarchy of **Autonomous Systems** (AS), e.g., ISPs, each owns routers and links. Decentralized administration, so routing done hierarchically.
- **Intra-AS routing:** Finds good path btwn two routers within AS. Commonly used protocols: RIP, OSPF
- **Inter-AS routing (not covered)** Handles the interfaces between ASs, standard protocol: BGP.

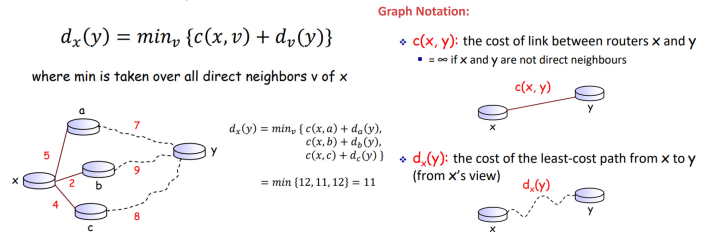
Intra-AS Routing

- Abstractly view a network of routers as a graph, vertices are routers, edges are physical links between routers.
- Associate cost to each link. (cost = 1, or inversely related to bandwidth, or related to congestion)

- **Routing:** find least cost path btwn two vertices in graph.
- **Link state Algorithms:** Centralised routing algo, all routers have complete knowledge of network topology and link costs. Routers periodically broadcast link costs to each other.
- Use Dijkstra algorithm compute least cost path locally! (using global map).
- **Distance vector Algorithms:** Decentralised routing algo, Routers know physically-connected neighbors and link costs to neighbors.
- Routers exchange “local views” with neighbors, update own “local views”. Iterative computation: Swap local view with direct neighbours, Update own view, Repeat till no further change.

Distance Vector Algo (Bellman Ford)

Bellman-Ford Equation



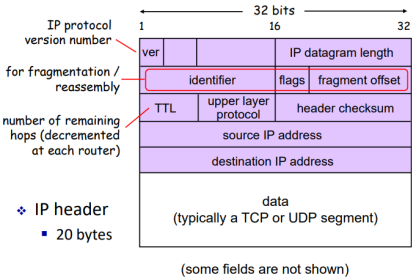
- $d_x(y) = \min_v \{c(x, v) + d_v(y)\}$
- To find **least cost path**, x needs to know cost from each of its direct neighbour to y. Each neighbour v sends its distance vector (y, k) to x, telling x that the cost from v to y is k.
- Every router, x, y, z, sends its distance vectors to directly connected neighbors. When x finds y is advertising cheaper path to z than known, x update distance vector to z accordingly and note down all packets for z should be sent to y. Info used to create forwarding table of x.
- After every router exchanged several rounds of updates with direct neighbors, all routers will know least-cost paths to all other routers.

RIP (Routing Information Protocol)

- **RIP** implements the DV algorithm. Uses hop count as the cost metric (insensitive to network congestion).
- Exchange routing table every 30 seconds over UDP port 520.
- “Self-repair”: if no update from a neighbour router for 3 minutes, assume neighbour failed.

Internet Protocol (IP): IPv4

IPv4 Datagram Format

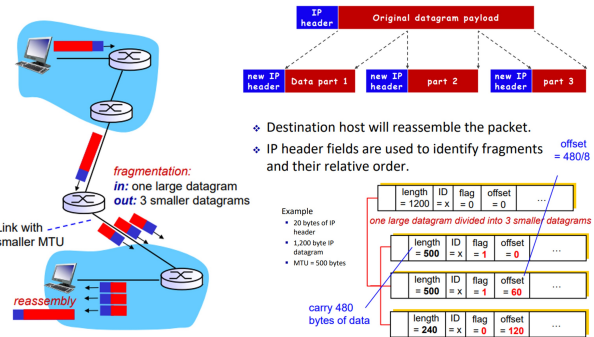


- Length: of IP datagram + 20b header. Identifier, flags, fragment offset: support fragmentation & reassembly
- TTL: Prevent infinite circulation. Upper layer protocol: Only used at final dest, determine if UDP/TCP (for Internet). Checksum also uses 1s complement.
- **IPv6:** 40b header with 128b IP addr.

IP Fragmentation & Reassembly

- Different links, different MTU (Max Transfer Unit, max amt of data link-level frame can carry). “Too large” IP datagrams may be fragmented by routers.

IP Fragmentation Illustration:



- **Flag**(frag flag) is set to 1 if next fragment from same segment, 0 if this is the last fragment. **Offset** is in expressed in unit of 8-bytes.

Internet Control Message Protocol

- **ICMP:** used by hosts & routers to communicate network-level information.
- **Error reporting:** unreachable host / network / port / protocol.
- Echo request/reply (used by ping).
- ICMP messages carried in IP datagrams, ICMP header starts after IP header.

ICMP Type and Code

- **ICMP header:** Type + Code + Checksum + others.

Type	Code	Description
8	0	echo request (ping)
0	0	echo reply (ping)
3	1	dest host unreachable
3	3	dest port unreachable
11	0	TTL expired
12	0	bad IP header

Selected ICMP Type and subtype (Code)